

Análise de Algoritmos Aplicada à Inserção sem Duplicatas em CSV

Comparação entre busca sequencial (iterativa e recursiva) e MergeSort com busca binária (iterativos e recursivos): análise teórica, demonstração por indução e validação experimental

Autores: Vitor Kenzo Matsunaga, Gustavo Alves, Matheus Brunes

Disciplina: Análise de Algoritmos — Relatório 4

Professor: Zoe Roberto

Repositório Git público

<https://github.com/owhska/insercao-sem-duplicatas-csv>

Setembro de 2026

Sumário

1	Introdução	3
2	Base de dados	3
2.1	Descrição	3
2.2	Coluna utilizada como chave de unicidade	4
2.3	Verificação dos requisitos	4
3	Descrição da solução implementada	4
4	Explicação do funcionamento do programa	5
5	Estruturas de dados utilizadas	5
6	Análise teórica da Solução 1	6
6.1	Hipóteses	6
6.2	Custo de uma inserção isolada	6
6.3	Complexidade assintótica	6
6.4	Comportamento quando N dobra	6
7	Dedução da função $T(N, Q)$	7
8	Demonstração por indução matemática	7
9	Análise das duas implementações da busca sequencial	8
9.1	Busca sequencial iterativa	8
9.2	Busca sequencial recursiva	8
9.3	Comparação teórica entre iterativa e recursiva	9
9.4	Custo total da Solução 1	9
10	Análise teórica da Solução 2	9
10.1	Funcionamento	9
10.2	MergeSort recursivo	10
10.3	MergeSort iterativo	11
10.4	Busca binária iterativa e recursiva	11
10.5	Custo total da Solução 2	12
10.6	Comparação teórica entre as Soluções 1 e 2	12
11	Metodologia experimental	13
11.1	Ambiente	13
11.2	Preparação das entradas	13
11.3	Os experimentos	13
11.4	Execução nos quatro modos	14
12	Tabelas e gráficos dos experimentos	14
12.1	Solução 1: Experimento A	14
12.2	Solução 1: Experimento C	14

12.3	Solução 1: gráficos	15
12.4	Solução 1: Experimentos D e E	16
12.5	Solução 2: Experimento A	17
12.6	Solução 2: Experimento C	17
12.7	Solução 2: Experimentos D e E	18
12.8	Tempos de execução	18
13	Comparação entre os resultados teóricos e empíricos	19
13.1	Aderência quantitativa da Solução 1	19
13.2	Taxa de crescimento ao dobrar N	20
13.3	O desvio do Experimento D, e por que ele é esperado	20
13.4	Aderência quantitativa da Solução 2	20
13.5	O efeito da quase-ordenação do arquivo original	20
13.6	Comparação de tempo entre iterativa e recursiva	21
13.7	Crescimento do tempo	21
13.8	Solução 1 $\times$ Solução 2	22
14	Discussão dos resultados	22
15	Conclusão	23
16	Repositório Git	23
A	Código-fonte completo	24

1 Introdução

Este trabalho tem por objetivo implementar, em linguagem C, um serviço de importação de registros de bolsistas a partir de arquivos CSV, garantindo que nenhum valor da coluna utilizada como chave de unicidade seja cadastrado em duplicidade, e analisar rigorosamente o custo computacional dos algoritmos empregados na verificação de redundância. O programa recebe, pela linha de comando, o nome do arquivo CSV com os novos registros e o nome do arquivo CSV de destino; os registros válidos são inseridos ao final do destino.

A verificação de duplicidade foi implementada de duas maneiras, correspondentes às duas soluções pedidas no enunciado:

Solução 1

busca sequencial sobre as chaves já presentes no destino, em versão iterativa e recursiva;

Solução 2

carga das chaves em um vetor na memória principal, ordenação por MergeSort (versões iterativa e recursiva) e busca por duplicidade com busca binária (versões iterativa e recursiva).

Todas as implementações foram instrumentadas com contadores do número de comparações entre chaves (chamadas a `strcmp`). O objetivo central não é apenas mostrar que o programa funciona, mas confrontar previsões teóricas com evidências medidas. A análise parte de dois parâmetros: Q , a quantidade de registros já existentes no arquivo de destino no início da execução, e N , a quantidade de registros novos a inserir. Para a Solução 1, a função de custo deduzida é

$$T(N, Q) = N \cdot Q + \frac{N(N-1)}{2}, \quad (1)$$

demonstrada por indução matemática; para a Solução 2, o custo é $\Theta((N+Q)\lg(N+Q))$, com limites inferior e superior deduzidos para cada fase. Os experimentos foram conduzidos sobre o arquivo `bolsistas-2025.csv`, obtido do Portal de Dados Abertos (`dados.gov.br`), variando N de 10 a 718 com os dados reais e até 512.000 em um benchmark em memória, nos quatro modos de execução.

Adiantam-se os resultados principais. Na Solução 1, em todas as execuções em que a hipótese de chaves distintas é satisfeita, a diferença entre o número de comparações previsto pela Equação (1) e o medido foi exatamente zero, nos dois modos. Na Solução 2, todas as contagens ficaram dentro dos limites teóricos. Para 8.000 registros, a Solução 2 foi 60 vezes mais rápida que a Solução 1. As versões recursivas têm a mesma complexidade temporal das iterativas, mas a busca sequencial recursiva foi cerca de 3 vezes mais lenta e usa pilha proporcional ao tamanho do cadastro.

2 Base de dados

2.1 Descrição

O arquivo utilizado tem as seguintes características:

- 719 registros de dados, além da linha de cabeçalho; separador vírgula, sem aspas; codificação Latin-1;
- quatro colunas: `Nome` (nome do bolsista), `Modalidade` (PQ, Produtividade em Pesquisa, com 696 registros; DT, Produtividade em Desenvolvimento Tecnológico e Extensão Inovadora, com 23 registros), `Nível` (nível da bolsa: 1A, 1B, 1C, 1D, 2, A, B, C) e `Agência` (agência financiadora: CNPq, com 694 registros, e FAPEMIG, com 25 registros);
- 718 nomes distintos, com maior nome de 43 caracteres.

2.2 Coluna utilizada como chave de unicidade

A chave de unicidade escolhida é a coluna `Nome`. O cadastro principal deve conter cada bolsista uma única vez; um segundo registro com o mesmo nome é tratado como tentativa de recadastramento, ainda que `Modalidade`, `Nível` e `Agência` sejam diferentes. O próprio arquivo original contém uma violação dessa regra: o nome MARCELO MACHADO VIANA aparece duas vezes, uma como `DT/C/CNPq` e outra como `PQ/2/FAPEMIG`. Esse é exatamente o tipo de registro que o programa deve rejeitar, e ele é usado no Experimento D.

A escolha tem uma limitação conhecida: nomes de pessoas não são identificadores perfeitos, pois pode haver homônimos. Em uma aplicação real, um identificador como o CPF ou o número do processo seria preferível; o arquivo público, porém, não traz esse tipo de identificador, e o objeto de estudo deste trabalho é o custo da verificação de unicidade, que independe de qual coluna é usada como chave.

2.3 Verificação dos requisitos

1. **Coluna cujos valores devem ser únicos:** `Nome`, pelas razões acima.
2. **Valores não ordenados no arquivo original:** o arquivo está organizado em dois blocos, primeiro os 694 registros do `CNPq` e depois os 25 da `FAPEMIG`, cada bloco em ordem alfabética de nome. A coluna `Nome` como um todo, portanto, não está ordenada: na transição entre os blocos o nome volta ao início do alfabeto. O arquivo é, no entanto, *quase* ordenado, e essa característica tem efeito mensurável sobre o MergeSort (Seção 13.5). Para que os experimentos principais não dependessem dessa quase-ordenação, os arquivos de novos registros usados nos testes foram gerados embaralhando os registros com semente fixa (Seção 11.2).

3 Descrição da solução implementada

O programa é um único executável em C, `copiador`, invocado da seguinte forma:

```
./copiador <novos.csv> <destino.csv> [seq-iter|seq-rec|bin-iter|bin-rec] [estatisticas.csv]
```

novos.csv

arquivo de onde os novos registros são lidos (a primeira linha é o cabeçalho);

destino.csv

cadastro principal, ao final do qual os registros não redundantes são acrescentados. Se não existir, é criado com o cabeçalho do arquivo de novos registros;

modo parâmetro opcional que seleciona a implementação: `seq-iter` (padrão) e `seq-rec` para a Solução 1, `bin-iter` e `bin-rec` para a Solução 2;

estatisticas.csv

parâmetro opcional; se informado, recebe ao final uma linha com os registros lidos, inseridos e duplicados, as comparações da ordenação e da busca e os tempos de execução.

Os dois primeiros argumentos são os exigidos pelo enunciado. A Solução 1 é a estratégia mais direta possível: o programa mantém em memória um vetor com as chaves já presentes no destino e, para cada novo registro, percorre esse vetor comparando chave a chave até encontrar uma igualdade ou esgotá-lo. A Solução 2 ordena as chaves com MergeSort e verifica a duplicidade com busca binária, de forma detalhada na Seção 10.

Três decisões de implementação merecem registro. Primeiro, o destino é carregado por completo antes de começar a inserção, o que estabelece o valor de Q ; sem essa etapa, o programa reinseriria chaves já cadastradas a cada nova execução. Segundo, a comparação considera exclusivamente a coluna `Nome`. Terceiro, uma redundância não interrompe a execução: ao encontrar uma chave repetida, o

programa emite a mensagem `ERRO: valor duplicado na chave, registro ignorado: <nome>` e segue para o próximo registro. Um registro é rejeitado quando sua chave já está no destino ou quando já apareceu em um registro anterior do próprio arquivo de novos registros. As quatro implementações produzem arquivos de destino idênticos byte a byte em todos os experimentos, o que foi verificado por *hash* MD5.

4 Explicação do funcionamento do programa

O fluxo de execução é o seguinte:

1. **Validação dos argumentos.** Se o número de argumentos estiver fora do esperado ou o modo for inválido, o programa imprime a forma de uso em `stderr` e encerra com código de erro.
2. **Leitura dos novos registros.** O arquivo de novos registros é lido por completo; o cabeçalho é guardado, e para cada linha são armazenadas a linha inteira e a chave extraída pela função `extrair_chave`.
3. **Preparação do destino.** Se o destino não existir ou estiver vazio, é criado com o cabeçalho; caso contrário, a função `garantir_quebra_final` assegura que ele termina em quebra de linha.
4. **Carga do cadastro existente.** A função `carregar_chaves_destino` lê o destino, ignora o cabeçalho e as linhas vazias e insere as chaves no vetor em memória. O valor retornado é Q .
5. **Verificação e inserção.** Na Solução 1, cada chave nova é procurada por busca sequencial (iterativa ou recursiva); se não for encontrada, a linha é gravada no destino e a chave passa a integrar o vetor. Na Solução 2, os vetores são ordenados por MergeSort e cada chave é verificada por busca binária (Seção 10).
6. **Relatório e estatísticas.** Ao final, o programa imprime um resumo na saída padrão e, se solicitado, acrescenta uma linha ao arquivo de estatísticas.

A instrumentação é feita por dois contadores globais: `comp_ordenacao`, incrementado a cada comparação da intercalação do MergeSort, e `comp_busca`, incrementado a cada comparação das buscas. Essa é a definição operacional de comparação adotada neste trabalho: cada chamada a `strcmp` entre duas chaves.

5 Estruturas de dados utilizadas

A estrutura central é um vetor dinâmico de cadeias de caracteres:

Listing 1: Estrutura de dados *Vetor*

```
1 typedef struct {
2     char **itens;
3     size_t qtd;
4     size_t capacidade;
5 } Vetor;
```

O campo `itens` é um vetor de ponteiros; cada posição aponta para uma cadeia alocada com o tamanho exato. O campo `qtd` guarda quantos itens estão armazenados, e `capacidade`, quantos caberiam sem realocação. O programa usa quatro vetores: as chaves do destino, as linhas dos novos registros, as chaves dos novos registros e, na Solução 2, uma cópia de ponteiros para as chaves novas, que é ordenada.

Crescimento amortizado. Quando `qtd` alcança `capacidade`, a função `vetor_inserir` dobra a capacidade com `realloc` (16 na primeira alocação). O dobramento garante que o custo total das realocações ao longo de N inserções seja $O(N)$, isto é, $O(1)$ amortizado por inserção; o crescimento do vetor, portanto, não contribui para o termo dominante do custo, que vem das comparações.

Vetor auxiliar do MergeSort. A intercalação usa um vetor auxiliar de ponteiros do mesmo tamanho do vetor ordenado, alocado uma única vez por ordenação.

6 Análise teórica da Solução 1

6.1 Hipóteses

A análise adota as seguintes hipóteses:

- (H1) a verificação de redundância é feita por busca sequencial;
- (H2) as N chaves a serem inseridas são distintas entre si e distintas das chaves do destino;
- (H3) o arquivo de destino inicia com Q registros.

A hipótese (H2) é a mais importante, pois torna a análise determinística: nenhuma busca encontra o que procura, e portanto toda busca percorre o vetor até o fim. Não existe caso médio nem melhor caso a considerar; há um único comportamento possível, que é o pior caso da busca sequencial.

6.2 Custo de uma inserção isolada

Considere a i -ésima inserção, com $i = 1, 2, \dots, N$. Antes dela, o vetor contém as Q chaves que já estavam no destino e as $i - 1$ chaves inseridas pelas iterações anteriores, totalizando $Q + i - 1$ elementos. Logo, a busca malsucedida executa

$$c_i = Q + i - 1 \quad (2)$$

comparações. O custo de cada inserção cresce linearmente com o número de inserções já feitas, porque o vetor contra o qual se compara cresce a cada passo.

6.3 Complexidade assintótica

Expandindo a Equação (1):

$$T(N, Q) = \frac{N^2}{2} + N \left(Q - \frac{1}{2} \right).$$

Para Q constante, o termo dominante é $N^2/2$. Para o limite superior, tomando $N \geq \max(Q, 2)$, tem-se $NQ \leq N^2$ e $N(N - 1)/2 \leq N^2/2$, logo

$$T(N, Q) \leq N^2 + \frac{N^2}{2} = \frac{3}{2}N^2,$$

o que satisfaz $O(N^2)$ com $c = 3/2$ e $n_0 = \max(Q, 2)$. Para o limite inferior, para $N \geq 2$ vale $N - 1 \geq N/2$, logo

$$T(N, Q) \geq \frac{N(N - 1)}{2} \geq \frac{N^2}{4},$$

o que satisfaz $\Omega(N^2)$ com $c = 1/4$ e $n_0 = 2$. Das duas condições segue $T(N) = \Theta(N^2)$. Essa classificação não muda se Q crescer proporcionalmente a N : se $Q = \alpha N$, então $T = (\alpha + 1/2)N^2 - N/2$, que continua sendo $\Theta(N^2)$, apenas com uma constante multiplicativa maior.

6.4 Comportamento quando N dobra

Com $Q = 0$, a razão entre o custo de $2N$ e o de N chaves é

$$\frac{T(2N)}{T(N)} = \frac{2N(2N - 1)}{N(N - 1)} = \frac{4N - 2}{N - 1}. \quad (3)$$

Tomando o limite, $\lim_{N \rightarrow \infty} \frac{4N-2}{N-1} = 4$: ao dobrar a quantidade de chaves, o número de comparações quadruplica. A convergência se dá por cima, e a distância até 4, que vale $\frac{2}{N-1}$, cai aproximadamente pela metade a cada duplicação de N (passa a $\frac{2}{2N-1}$). O termo NQ , quando presente, não altera esse limite: para Q fixo ele é linear em N e, portanto, assintoticamente desprezível diante do termo quadrático; seu efeito é apenas retardar a convergência da razão para 4.

7 Dedução da função $T(N, Q)$

O total de comparações é a soma dos custos individuais das N inserções. Usando a Equação (2):

$$\begin{aligned}
 T(N, Q) &= \sum_{i=1}^N c_i = \sum_{i=1}^N (Q + i - 1) \\
 &= \sum_{i=1}^N Q + \sum_{i=1}^N (i - 1) \\
 &= N \cdot Q + \sum_{j=0}^{N-1} j \quad (j = i - 1) \\
 &= N \cdot Q + \frac{(N-1)N}{2} \\
 &= N \cdot Q + \frac{N(N-1)}{2}.
 \end{aligned} \tag{4}$$

A fórmula tem uma leitura direta: o termo $N \cdot Q$ é o custo de comparar cada uma das N chaves novas contra as Q que já estavam no destino (um retângulo de N por Q comparações), e o termo $N(N-1)/2$ é o custo de comparar as chaves novas entre si: são $\binom{N}{2}$ pares não ordenados, cada um comparado exatamente uma vez.

Casos particulares. Para $Q = 0$, a fórmula se reduz a $T(N, 0) = N(N-1)/2$. Para $Q = 1$, ocorre a simplificação

$$T(N, 1) = N + \frac{N(N-1)}{2} = \frac{N(N+1)}{2}. \tag{5}$$

8 Demonstração por indução matemática

Demonstra-se, por indução sobre N , a validade da Equação (1). Seja $Q \geq 0$ fixo e defina-se a proposição

$$P(N) : T(N, Q) = N \cdot Q + \frac{N(N-1)}{2}.$$

Caso base. Para $N = 1$, há uma única inserção; no momento em que ela ocorre, o vetor contém apenas os Q registros iniciais, e a busca, malsucedida por (H2), percorre todos eles. Logo $T(1, Q) = Q$. Avaliando a fórmula em $N = 1$: $1 \cdot Q + \frac{1 \cdot 0}{2} = Q$. Os dois valores coincidem, logo $P(1)$ é verdadeira. (A fórmula também vale trivialmente para $N = 0$, o que permite ancorar a indução em $N = 0$ caso se prefira.)

Hipótese de indução. Suponha que, para algum $k \geq 1$, $P(k)$ seja verdadeira, isto é, $T(k, Q) = kQ + \frac{k(k-1)}{2}$.

Passo indutivo. Considere a inserção de $k+1$ chaves distintas. As k primeiras inserções são idênticas às do caso com k chaves, custando, por hipótese de indução, $T(k, Q)$. Resta o custo da $(k+1)$ -ésima inserção: o vetor contém então $Q + k$ elementos, e como a nova chave é distinta de todos eles, por (H2),

a busca percorre o vetor inteiro, custando $Q + k$ comparações. Logo:

$$\begin{aligned}
 T(k+1, Q) &= T(k, Q) + (Q + k) \\
 &= kQ + \frac{k(k-1)}{2} + Q + k \\
 &= (k+1)Q + \frac{k(k-1) + 2k}{2} \\
 &= (k+1)Q + \frac{k^2 + k}{2} \\
 &= (k+1)Q + \frac{(k+1)k}{2} \\
 &= (k+1)Q + \frac{(k+1)((k+1)-1)}{2},
 \end{aligned}$$

que é precisamente a fórmula avaliada em $N = k + 1$. Portanto $P(k + 1)$ é verdadeira.

Conclusão. Verificou-se que $P(1)$ é verdadeira e que, para todo $k \geq 1$, $P(k) \Rightarrow P(k + 1)$. Pelo princípio da indução matemática,

$$T(N, Q) = N \cdot Q + \frac{N(N-1)}{2} \quad \text{para todo } N \geq 1,$$

sob as hipóteses (H1), (H2) e (H3). ■

Cabe sublinhar o que a demonstração estabelece e o que não estabelece: ela prova que a fórmula descreve exatamente o número de comparações quando as chaves inseridas são distintas, mas nada afirma sobre execuções com redundâncias; nesses casos, uma busca pode terminar antes de percorrer o vetor todo. O Experimento D (Seção 13.3) explora justamente essa fronteira.

9 Análise das duas implementações da busca sequencial

9.1 Busca sequencial iterativa

Listing 2: Busca sequencial iterativa

```

1 int busca_sequencial_iterativa(char **v, size_t n, const char *chave) {
2     size_t i;
3     for (i = 0; i < n; i++) {
4         comp_busca++;
5         if (strcmp(v[i], chave) == 0) return 1;
6     }
7     return 0;
8 }

```

Análise. Tempo no pior caso $\Theta(n)$; no melhor caso, $\Theta(1)$; no caso médio, $\Theta(n)$. Espaço auxiliar $\Theta(1)$: apenas a variável de controle i é usada, independentemente do tamanho do vetor. Espaço total $\Theta(n)$, correspondente ao próprio cadastro.

9.2 Busca sequencial recursiva

Listing 3: Busca sequencial recursiva

```

1 static int busca_sequencial_rec_aux(char **v, size_t n, const char *chave, size_t i) {
2     if (i >= n) return 0;
3     comp_busca++;
4     if (strcmp(v[i], chave) == 0) return 1;
5     return busca_sequencial_rec_aux(v, n, chave, i + 1);
6 }
7

```

```

8 int busca_sequencial_recursiva(char **v, size_t n, const char *chave) {
9     return busca_sequencial_rec_aux(v, n, chave, 0);
10 }

```

Análise. As complexidades de tempo são idênticas às da versão iterativa: $\Theta(n)$ no pior e no caso médio, $\Theta(1)$ no melhor caso. A diferença está no espaço: cada chamada recursiva empilha uma nova moldura de ativação, e nenhuma é liberada antes do fim da busca, de modo que a profundidade da pilha chega a $n + 1$ e o espaço auxiliar é $\Theta(n)$. O espaço total permanece $\Theta(n) + \Theta(n) = \Theta(n)$, mas com uma constante maior que a da versão iterativa.

Chamada de cauda. A chamada recursiva é a última operação da função (chamada de cauda). Com `-O2`, o GCC transforma esse tipo de chamada em salto e compila a função recursiva como um laço, sem empilhar molduras, o que foi confirmado inspecionando o código de máquina gerado. Para que a versão recursiva medida seja de fato recursiva, o programa é compilado com `-fno-optimize-sibling-calls` (Seção 11.1).

9.3 Comparação teórica entre iterativa e recursiva

Tabela 1: Comparação teórica entre as implementações iterativa e recursiva da busca sequencial

Critério	Iterativa	Recursiva
Tempo pior caso	$\Theta(n)$	$\Theta(n)$
Tempo melhor caso	$\Theta(1)$	$\Theta(1)$
Espaço auxiliar	$\Theta(1)$	$\Theta(n)$
Risco de <i>stack overflow</i>	Não	Sim (n grande)
<i>Overhead</i> por elemento	Nenhum	Uma chamada de função
Legibilidade	Alta	Média

Ambas têm a mesma complexidade temporal assintótica $\Theta(n)$, mas a iterativa é espacialmente superior ($\Theta(1)$ contra $\Theta(n)$), sem o risco de estouro de pilha em cadastros muito grandes.

9.4 Custo total da Solução 1

Como as duas implementações incrementam o contador exatamente uma vez por chamada a `strcmp`, percorrem o vetor na mesma ordem e param no mesmo ponto, o custo total da Solução 1, medido em número de comparações, é idêntico nos dois modos e segue a mesma função $T(N, Q)$ deduzida na Seção 7, com complexidade $\Theta(N(N + Q))$. A diferença entre os modos manifesta-se apenas no tempo de execução, devido ao custo das chamadas de função, e no consumo de pilha (Seção 13.6). O espaço total é $\Theta(N + Q)$, dominado pelo vetor de chaves e pelas linhas carregadas.

10 Análise teórica da Solução 2

Em toda esta seção, $\lg$ denota logaritmo na base 2 e o custo é medido em comparações entre chaves.

10.1 Funcionamento

A Solução 2 funciona em duas fases.

Ordenação. O vetor D com as Q chaves do destino é ordenado por MergeSort. Um segundo vetor S , com ponteiros para as N chaves do arquivo de novos registros, também é ordenado por MergeSort. As duas versões do MergeSort usam o mesmo procedimento de intercalação, que é *estável*: em caso de

empate, o elemento da metade esquerda é copiado primeiro. A versão recursiva é a clássica *top-down*; a iterativa é a *bottom-up*, que intercala blocos de largura 1, 2, 4, ... até cobrir o vetor.

Verificação. Para cada novo registro r , na ordem do arquivo:

1. uma busca binária em D verifica se a chave já existe no destino;
2. se não existe, uma busca binária em S localiza a *primeira* ocorrência da chave em S . Como a ordenação é estável, essa primeira ocorrência corresponde ao registro mais antigo com aquela chave. Se ela não é o próprio r (comparação de ponteiros, sem `strcmp`), então r repete a chave de um registro anterior e é rejeitado.

Essa organização evita inserir chaves no meio de um vetor ordenado, o que exigiria deslocar elementos a cada inserção e custaria $\Theta(N + Q)$ movimentações por registro.

Listing 4: Intercalação estável e MergeSort recursivo e iterativo

```

1 static void intercalar(char **v, char **aux, size_t ini, size_t meio, size_t fim) {
2     size_t i = ini, j = meio, k = ini;
3     while (i < meio && j < fim) {
4         comp_ordenacao++;
5         if (strcmp(v[i], v[j]) <= 0) aux[k++] = v[i++];
6         else aux[k++] = v[j++];
7     }
8     while (i < meio) aux[k++] = v[i++];
9     while (j < fim) aux[k++] = v[j++];
10    memcpy(v + ini, aux + ini, (fim - ini) * sizeof(char *));
11 }
12
13 static void mergesort_rec_aux(char **v, char **aux, size_t ini, size_t fim) {
14     size_t meio;
15     if (fim - ini < 2) return;
16     meio = ini + (fim - ini) / 2;
17     mergesort_rec_aux(v, aux, ini, meio);
18     mergesort_rec_aux(v, aux, meio, fim);
19     intercalar(v, aux, ini, meio, fim);
20 }
21
22 void mergesort_iterativo(char **v, size_t n) {
23     char **aux;
24     size_t largura, ini, meio, fim;
25     if (n < 2) return;
26     aux = alocar_auxiliar(n);
27     for (largura = 1; largura < n; largura *= 2) {
28         for (ini = 0; ini + largura < n; ini += 2 * largura) {
29             meio = ini + largura;
30             fim = (ini + 2 * largura < n) ? ini + 2 * largura : n;
31             intercalar(v, aux, ini, meio, fim);
32         }
33     }
34     free(aux);
35 }

```

10.2 MergeSort recursivo

A intercalação de dois blocos ordenados com a e b elementos faz no máximo $a + b - 1$ comparações (cada comparação coloca um elemento na saída, e o último sai sem comparação) e no mínimo $\min(a, b)$ (quando um bloco se esgota primeiro). O MergeSort recursivo divide n em $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil$, logo o número de comparações no pior caso satisfaz

$$W(1) = 0, \quad W(n) = W(\lfloor n/2 \rfloor) + W(\lceil n/2 \rceil) + n - 1,$$

cujas soluções exatas são $W(n) = n \lceil \lg n \rceil - 2^{\lceil \lg n \rceil} + 1$. O melhor caso satisfaz $B(n) = B(\lfloor n/2 \rfloor) + B(\lceil n/2 \rceil)$.

Proposição 1. Para $n = 2^k$, $k \geq 0$: $W(n) = n \lg n - n + 1$ e $B(n) = \frac{n}{2} \lg n$.

Demonstração. Por indução em k . Base $k = 0$: $W(1) = 0 = 1 \cdot 0 - 1 + 1$ e $B(1) = 0 = \frac{1}{2} \cdot 0$. Passo: supondo válido para $n = 2^k$,

$$W(2n) = 2W(n) + 2n - 1 = 2(n \lg n - n + 1) + 2n - 1 = 2n \lg n + 1 = 2n \lg(2n) - 2n + 1,$$

$$B(2n) = 2B(n) + n = n \lg n + n = \frac{2n}{2} \lg(2n). \quad \square$$

Como $\frac{n}{2} \lg n = B(n) \leq C(n) \leq W(n) \leq n \lg n$ para potências de 2 (e com ajuste de constantes para n geral), o MergeSort faz $\Theta(n \lg n)$ comparações em *qualquer* entrada. O número exato de comparações depende da ordem dos dados: entradas já ordenadas se aproximam de $B(n)$ e entradas aleatórias, de $W(n)$.

Espaço. A intercalação usa um vetor auxiliar de n ponteiros: $\Theta(n)$. A recursão tem profundidade $\lceil \lg n \rceil$, acrescentando $\Theta(\lg n)$ de pilha. Espaço auxiliar total: $\Theta(n) + \Theta(\lg n) = \Theta(n)$.

10.3 MergeSort iterativo

A versão *bottom-up* faz $\lceil \lg n \rceil$ passadas, com larguras de bloco $1, 2, 4, \dots$. Em cada passada, cada elemento participa de no máximo uma intercalação, logo uma passada faz no máximo $n - 1$ comparações; e em cada passada completa (blocos cheios) faz no mínimo cerca de $n/2$. O total fica, portanto, entre aproximadamente $\frac{n}{2} \lg n$ e $n \lceil \lg n \rceil$: $\Theta(n \lg n)$. Para n potência de 2 as duas versões fazem exatamente as mesmas intercalações; para outros valores de n os blocos são divididos de forma diferente e as contagens diferem ligeiramente, o que se observa nos experimentos. Os limites exatos da versão iterativa foram calculados simulando a sequência de intercalações (funções `ms_melhor_bottomup` e `ms_pior_bottomup` em `analisar.py`).

Espaço. O mesmo vetor auxiliar de n ponteiros, $\Theta(n)$, e apenas $\Theta(1)$ de variáveis de controle, sem pilha de recursão.

10.4 Busca binária iterativa e recursiva

Listing 5: Busca binária iterativa e recursiva (limite inferior)

```

1 size_t busca_binaria_iterativa(char **v, size_t n, const char *chave) {
2     size_t lo = 0, hi = n, meio;
3     while (lo < hi) {
4         meio = lo + (hi - lo) / 2;
5         comp_busca++;
6         if (strcmp(v[meio], chave) < 0) lo = meio + 1;
7         else hi = meio;
8     }
9     return lo;
10 }
11
12 static size_t busca_binaria_rec_aux(char **v, const char *chave, size_t lo, size_t hi) {
13     size_t meio;
14     if (lo >= hi) return lo;
15     meio = lo + (hi - lo) / 2;
16     comp_busca++;
17     if (strcmp(v[meio], chave) < 0) return busca_binaria_rec_aux(v, chave, meio + 1, hi);
18     return busca_binaria_rec_aux(v, chave, lo, meio);
19 }

```

As duas versões retornam a posição da primeira chave maior ou igual à procurada. Na busca em D , se essa posição é válida, faz-se mais uma comparação de igualdade, também contabilizada.

Tempo. A cada iteração, um intervalo de tamanho s passa a ter tamanho $\lfloor s/2 \rfloor$ ou $\lfloor (s-1)/2 \rfloor$. Logo o número de iterações (e de comparações) de uma busca em n elementos está entre

$$b_{\min}(n) = \lfloor \lg(n+1) \rfloor \quad \text{e} \quad b_{\max}(n) = \lfloor \lg n \rfloor + 1,$$

que diferem de no máximo uma unidade: a busca binária é $\Theta(\lg n)$ em qualquer caso, pois a versão implementada nunca termina antes de reduzir o intervalo a zero. As versões iterativa e recursiva fazem as mesmas comparações.

Espaço. A iterativa usa $\Theta(1)$. A recursiva empilha uma moldura por iteração, com profundidade $\Theta(\lg n)$; mesmo para $n = 10^9$ isso seria cerca de 30 molduras, sem risco de estouro.

Tabela 2: Comparação teórica entre as implementações iterativa e recursiva da Solução 2

Critério	Iterativa	Recursiva
MergeSort: tempo (todos os casos)	$\Theta(n \lg n)$	$\Theta(n \lg n)$
MergeSort: espaço auxiliar	$\Theta(n)$	$\Theta(n) + \Theta(\lg n) = \Theta(n)$
Busca binária: tempo	$\Theta(\lg n)$	$\Theta(\lg n)$
Busca binária: espaço auxiliar	$\Theta(1)$	$\Theta(\lg n)$ (pilha)

10.5 Custo total da Solução 2

Com Q chaves no destino e N novos registros de chaves distintas, e notando por $C_{\text{ms}}(n)$ as comparações do MergeSort,

$$T_2(N, Q) = C_{\text{ms}}(Q) + C_{\text{ms}}(N) + \sum_{r=1}^N [b_D + e + b_S],$$

em que $b_D \in [b_{\min}(Q), b_{\max}(Q)]$ é o custo da busca em D , $e \in \{0, 1\}$ é a comparação de igualdade e $b_S \in [b_{\min}(N), b_{\max}(N)]$ é o custo da busca em S . Assim,

$$T_2(N, Q) = \Theta(Q \lg Q) + \Theta(N \lg N) + \Theta(N \lg Q) + \Theta(N \lg N) = \Theta((N + Q) \lg(N + Q)).$$

Para $Q = 0$, as duas fases somam aproximadamente $2N \lg N$ comparações.

Espaço. Vetores D e S ($Q + N$ ponteiros), vetor auxiliar do MergeSort ($\max(Q, N)$ ponteiros) e as linhas dos novos registros: $\Theta(N + Q)$. As versões recursivas acrescentam apenas $\Theta(\lg(N + Q))$ de pilha.

10.6 Comparação teórica entre as Soluções 1 e 2

Tabela 3: Comparação teórica entre as soluções, com $n = N + Q$

	Solução 1	Solução 2
Tempo (comparações)	$NQ + N(N - 1)/2 = \Theta(Nn)$	$\Theta(n \lg n)$
Tempo, $Q = 0$	$\approx N^2/2$	$\approx 2N \lg N$
Espaço total	$\Theta(n)$	$\Theta(n)$, com vetor auxiliar extra
Pilha (versões recursivas)	$\Theta(n)$	$\Theta(\lg n)$

A Solução 2 é assintoticamente superior no tempo. No espaço ambas são lineares, mas a Solução 2 precisa de um vetor auxiliar a mais; em compensação, suas versões recursivas usam pilha logarítmica, enquanto a busca sequencial recursiva usa pilha linear. Igualando $N^2/2$ a $2N \lg N$, obtém-se $N \approx 16$: abaixo disso a Solução 1 pode fazer menos comparações.

11 Metodologia experimental

11.1 Ambiente

Os experimentos foram executados em Linux x86-64 com GCC 13.3.0, com compilação por

```
gcc -O2 -fno-optimize-sibling-calls -Wall -Wextra -std=c99 -o src/copiador src/copiador.c src/
algoritmos.c
gcc -O2 -fno-optimize-sibling-calls -Wall -Wextra -std=c99 -o src/bench src/bench.c src/
algoritmos.c
```

O código também compila no Windows (MSYS2). O nível de otimização é irrelevante para a contagem de comparações, que é uma contagem de eventos discretos do algoritmo; ele é relevante, porém, para o tempo e para o uso de pilha, e por isso a opção `-fno-optimize-sibling-calls` é usada para impedir que as recursões de cauda sejam convertidas em laços. Toda a bateria é executada pelo script `rodar_experimentos.sh`, e a análise, pelo `analisar.py`.

11.2 Preparação das entradas

O arquivo `bolsistas-2025.csv` contém 719 registros e 718 nomes distintos; há exatamente uma redundância real, o nome MARCELO MACHADO VIANA. O script `preparar_entradas.py` extrai os 718 registros de nomes distintos, embaralha-os com semente fixa (2026) e gera os arquivos `novos_N.csv` com os N primeiros registros, para $N \in \{10, 20, 40, 80, 160, 320, 640, 718\}$, o que assegura o cumprimento da hipótese (H2). Os arquivos `previos_Q.csv`, usados como destino inicial, contêm os registros das posições 160 a $160 + Q - 1$ da mesma lista embaralhada, e portanto são reais e distintos dos 160 primeiros.

11.3 Os experimentos

Experimento A ($Q = 0$)

Destino iniciando vazio, para os oito valores de N . Na Solução 1, testa o termo quadrático isoladamente; previsão: $N(N - 1)/2$.

Experimento C (Q variável)

$N = 160$ fixo, com $Q \in \{0, 10, 20, 40, 80, 160\}$. Na Solução 1, isola o termo $N \cdot Q$: a fórmula prevê dependência linear em Q , com inclinação 160.

Experimento D (arquivo completo)

Arquivo `bolsistas-2025.csv` integral, na ordem original, com a redundância real presente, e destino vazio. A hipótese (H2) é violada de propósito.

Experimento E (reexecução)

O programa é executado novamente sobre o destino preenchido pelo Experimento D ($Q = 718$). Todos os 719 registros são redundantes e nada é inserido.

Tempos com dados reais

Para cada N do Experimento A e cada modo, 15 execuções completas do programa; usa-se a mediana.

Benchmark em memória

O programa `bench` executa as duas soluções sem leitura e escrita de arquivos, com chaves geradas a partir dos nomes reais (acrescidos de um sufixo numérico para torná-las distintas) e embaralhadas. Solução 1: N de 1.000 a 8.000; Solução 2: N de 1.000 a 512.000. Usa-se a mediana de 15 repetições. Esse experimento é necessário porque, com apenas 718 registros, os tempos da Solução 2 ficam na casa de décimos de milissegundo e são dominados por leitura e escrita.

11.4 Execução nos quatro modos

Cada experimento foi executado nos quatro modos (seq-iter, seq-rec, bin-iter e bin-rec). Por modo, os Experimentos A e C somam 14 execuções sob a hipótese (H2), e D e E, 2 fora dela: ao todo, 64 execuções para contagem de comparações. A medição de tempo com dados reais somou mais 480 execuções (8 valores de N , 4 modos, 15 repetições). Todos os arquivos de entrada, destinos, saídas e estatísticas estão no diretório `experimentos/` do repositório.

12 Tabelas e gráficos dos experimentos

12.1 Solução 1: Experimento A

Tabela 4: Solução 1, Experimento A ($Q = 0$): comparações iterativa e recursiva

N	Comp. iter.	Comp. rec.	Previsto $T(N, 0)$	Diferença
10	45	45	45	0
20	190	190	190	0
40	780	780	780	0
80	3.160	3.160	3.160	0
160	12.720	12.720	12.720	0
320	51.040	51.040	51.040	0
640	204.480	204.480	204.480	0
718	257.403	257.403	257.403	0

A diferença é nula em todos os pontos, em ambos os modos de busca.

12.2 Solução 1: Experimento C

Tabela 5: Solução 1, Experimento C ($N = 160$ fixo): isolamento do termo $N \cdot Q$

Q	Comp. iter.	Comp. rec.	Previsto $T(160, Q)$	Diferença
0	12.720	12.720	12.720	0
10	14.320	14.320	14.320	0
20	15.920	15.920	15.920	0
40	19.120	19.120	19.120	0
80	25.520	25.520	25.520	0
160	38.320	38.320	38.320	0

Cada incremento de 10 em Q acrescenta exatamente 1.600 comparações, isto é, 160×10 , em ambos os modos.

12.3 Solução 1: gráficos

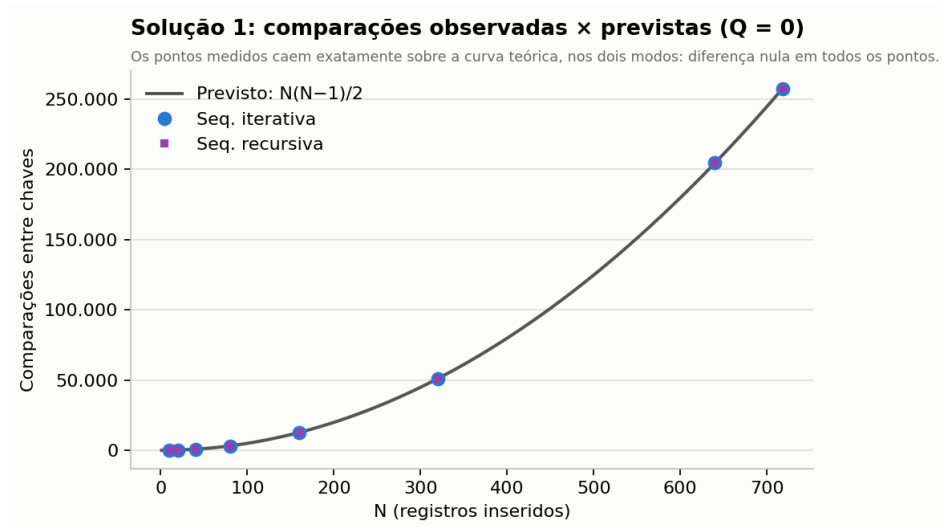


Figura 1: Comparações observadas (pontos) sobre a curva teórica $N(N-1)/2$ (linha), Experimento A. Os dois modos se sobrepõem e não há ponto fora da curva: a sobreposição é exata, não aproximada.

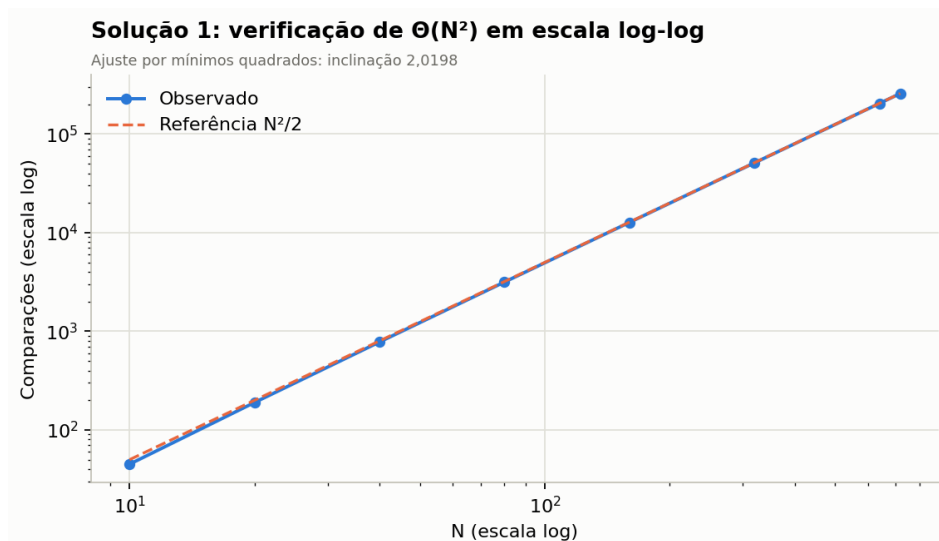


Figura 2: Os mesmos dados em escala log-log. O ajuste por mínimos quadrados sobre os dados medidos resulta em inclinação 2,0198, coerente com $\Theta(N^2)$.

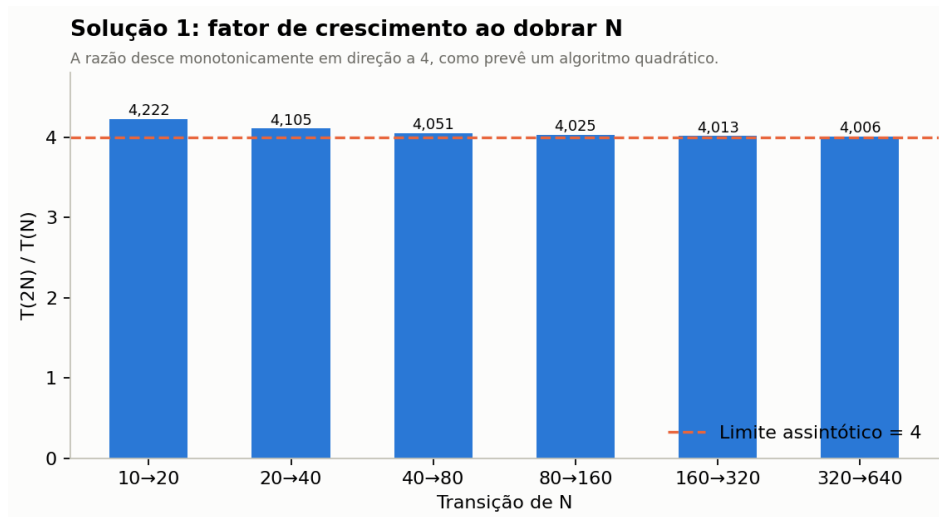


Figura 3: Razão $T(2N)/T(N)$ medida em cada duplicação de N . A razão decresce monotonicamente em direção a 4, exatamente como prevê a Equação (3).

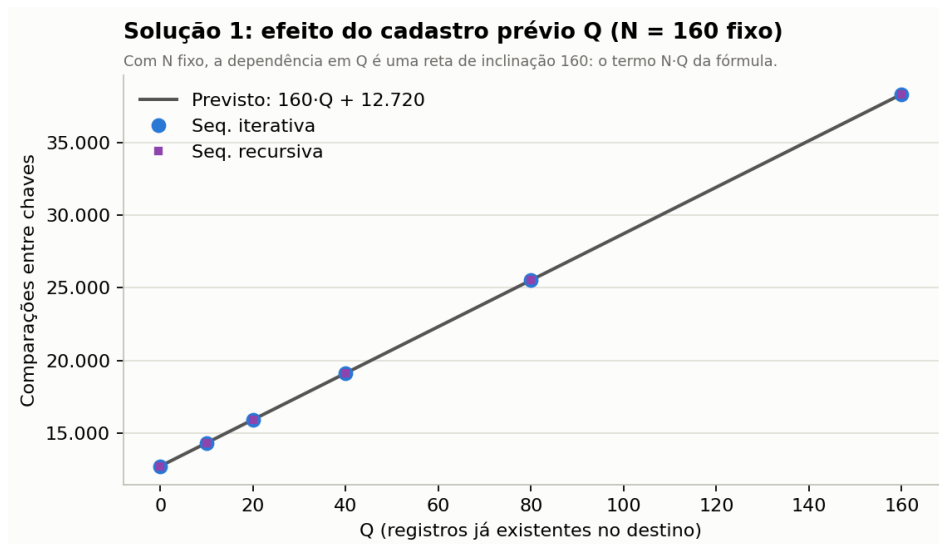


Figura 4: Experimento C. Com $N = 160$ fixo, as comparações crescem linearmente com Q , sobre a reta $160Q + 12.720$: confirmação direta do termo $N \cdot Q$ da fórmula.

12.4 Solução 1: Experimentos D e E

Tabela 6: Solução 1 com redundâncias reais, fora da hipótese ($H2$) (mesmos valores nos dois modos)

Experimento	Registros lidos	Inseridos	Redundantes	Comparações
D — arquivo completo, $Q = 0$	719	718	1	257.841
E — reexecução, $Q = 718$	719	0	719	258.559

O Experimento D insere os mesmos 718 nomes do último ponto do Experimento A, mas registra 438 comparações a mais, em ambos os modos de busca. A explicação está na Seção 13.3.

12.5 Solução 2: Experimento A

A Tabela 7 mostra as comparações de cada fase e a faixa teórica [mínimo, máximo] deduzida na Seção 10. A busca binária faz o mesmo número de comparações nos dois modos.

Tabela 7: Solução 2, Experimento A ($Q = 0$): comparações medidas e faixas teóricas

N	MS iterativo	faixa	MS recursivo	faixa	Busca binária	faixa
10	27	[15, 27]	22	[15, 25]	36	[30, 40]
20	67	[40, 73]	64	[40, 69]	90	[80, 100]
40	177	[100, 185]	167	[100, 177]	218	[200, 240]
80	420	[240, 449]	412	[240, 433]	514	[480, 560]
160	1.004	[560, 1.057]	966	[560, 1.025]	1.186	[1.120, 1.280]
320	2.343	[1.280, 2.433]	2.264	[1.280, 2.369]	2.690	[2.560, 2.880]
640	5.329	[2.880, 5.505]	5.173	[2.880, 5.377]	6.018	[5.760, 6.400]
718	6.024	[3.267, 6.227]	5.921	[3.267, 6.157]	6.876	[6.462, 7.180]

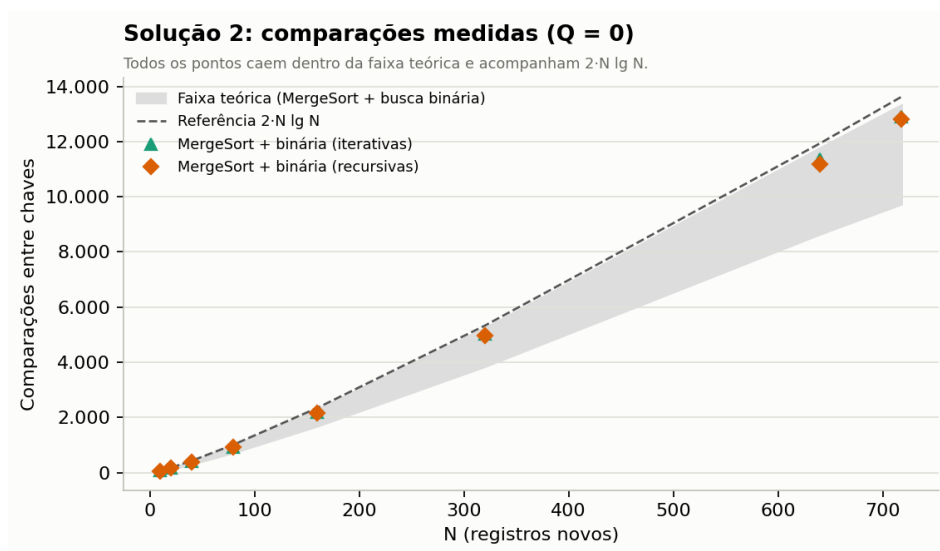


Figura 5: Solução 2, Experimento A: comparações totais medidas dentro da faixa teórica, próximas de $2N \lg N$.

12.6 Solução 2: Experimento C

Tabela 8: Solução 2, Experimento C ($N = 160$ fixo): comparações totais (MergeSort + busca binária)

Q	Iterativa	Recursiva	Solução 1 (para referência)
0	2.190	2.152	12.720
10	2.919	2.878	14.320
20	3.161	3.117	15.920
40	3.413	3.362	19.120
80	3.787	3.728	25.520
160	4.560	4.483	38.320

12.7 Solução 2: Experimentos D e E

Tabela 9: Solução 2 com o arquivo na ordem original (D) e na reexecução (E)

Experimento	MS iterativo	MS recursivo	Busca binária	Redundantes
A, $N = 718$ (embaralhado)	6.024	5.921	6.876	0
D, $N = 719$ (ordem original)	3.738	3.353	6.887	1
E, $N = 719$, $Q = 718$	7.471	6.694	7.604	719

12.8 Tempos de execução

Tabela 10: Benchmark em memória, $Q = 0$: tempo total em milissegundos (mediana de 15 repetições)

N	Seq. iterativa	Seq. recursiva	Bin. iterativa	Bin. recursiva
1.000	2,963	8,349	0,250	0,276
2.000	11,925	36,274	0,638	0,667
4.000	48,931	149,041	1,477	1,592
8.000	199,481	609,557	3,313	3,445
16.000	—	—	8,055	8,340
32.000	—	—	17,994	19,062
64.000	—	—	41,675	41,771
128.000	—	—	96,319	97,624
256.000	—	—	226,898	233,551
512.000	—	—	527,291	566,208

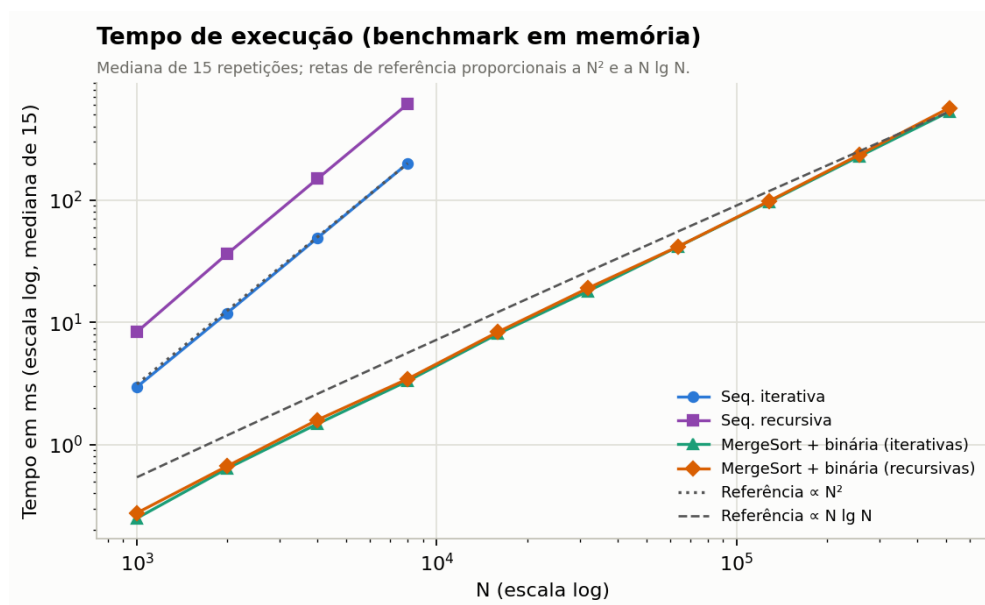


Figura 6: Tempos do benchmark em escala log-log, com retas de referência proporcionais a N^2 e a $N \lg N$.

Tabela 11: Tempo do programa com os arquivos reais, $Q = 0$, em milissegundos (mediana de 15 execuções; inclui a gravação no destino)

N	Seq. iterativa	Seq. recursiva	Bin. iterativa	Bin. recursiva
10	0,0025	0,0029	0,0050	0,0051
20	0,0059	0,0061	0,0075	0,0081
40	0,0126	0,0134	0,0143	0,0152
80	0,0256	0,0444	0,0248	0,0270
160	0,0845	0,1815	0,0548	0,0572
320	0,2815	0,7400	0,1137	0,1182
640	1,0394	2,9875	0,2334	0,2434
718	1,2809	3,7721	0,2633	0,2872

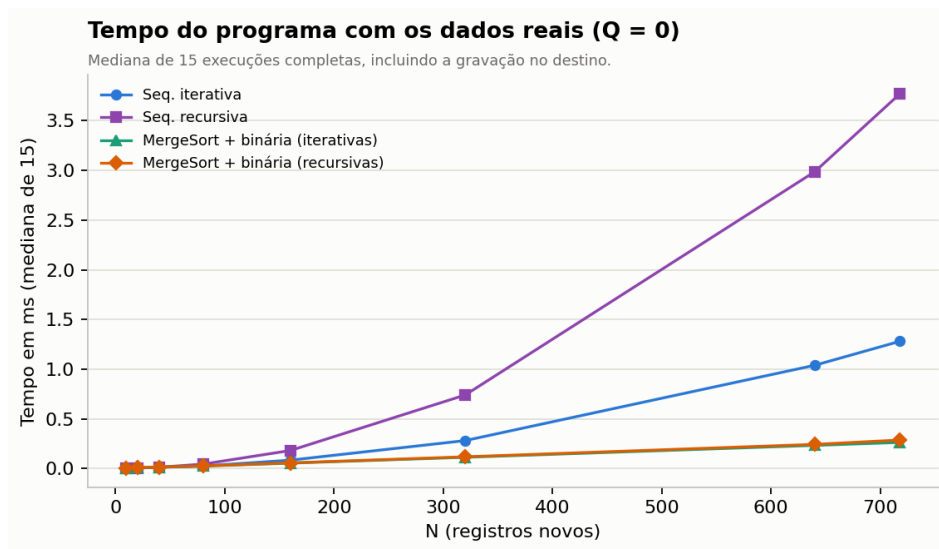


Figura 7: Tempo de execução medido com os dados reais. A partir de $N \approx 80$ a Solução 2 passa a ser mais rápida.

13 Comparação entre os resultados teóricos e empíricos

13.1 Aderência quantitativa da Solução 1

Nos Experimentos A e C, as 14 configurações em que a hipótese (H2) é satisfeita, executadas nos dois modos de busca (28 execuções), a diferença entre observado e previsto foi zero em todos os pontos, sem exceção. Não se trata de uma aproximação com erro pequeno, mas de coincidência exata dígito a dígito, inclusive em valores da ordem de $2,5 \times 10^5$ comparações.

Dois aspectos distintos da fórmula foram testados separadamente: o termo quadrático $N(N - 1)/2$ (Experimento A) e o termo linear $N \cdot Q$ (Experimento C, mantendo N fixo e variando Q : cada acréscimo de 10 registros prévios produziu exatamente 1.600 comparações adicionais).

13.2 Taxa de crescimento ao dobrar N

Tabela 12: Fator de crescimento observado ao dobrar N (Solução 1), comparado ao valor exato previsto

Transição	Razão medida	Previsto por $(4N - 2)/(N - 1)$	Diferença
10 $\rightarrow$ 20	4,2222	4,2222	0,0000
20 $\rightarrow$ 40	4,1053	4,1053	0,0000
40 $\rightarrow$ 80	4,0513	4,0513	0,0000
80 $\rightarrow$ 160	4,0253	4,0253	0,0000
160 $\rightarrow$ 320	4,0126	4,0126	0,0000
320 $\rightarrow$ 640	4,0063	4,0063	0,0000

A concordância é total, e o padrão de convergência também é o previsto: a razão nunca é exatamente 4 para N finito, aproxima-se de 4 por valores maiores, e a distância até 4, que vale $2/(N - 1)$, reduz-se aproximadamente pela metade a cada duplicação de N .

13.3 O desvio do Experimento D, e por que ele é esperado

Comparando com o último ponto do Experimento A, que insere os mesmos 718 nomes: $257.841 - 257.403 = 438$. O arquivo completo tem 719 registros, dos quais 718 são inserções e um é redundante. As 718 inserções custam, pela fórmula, $718 \times 717/2 = 257.403$ comparações. Resta o custo da busca pelo registro redundante: a segunda ocorrência de MARCELO MACHADO VIANA é o 705.º registro do arquivo; quando ela é lida, o vetor tem 704 nomes, mas a busca sequencial interrompe-se ao encontrar a igualdade na posição 438, onde o nome foi inserido, consumindo 438 comparações em vez de 704. Logo $257.403 + 438 = 257.841$, exatamente o valor medido, em ambos os modos de busca.

O Experimento E confirma a mesma mecânica em regime extremo: com o destino contendo os 718 nomes e nenhuma inserção possível, cada uma das 719 buscas é bem-sucedida e termina na posição em que o nome está armazenado. Como esses nomes ocupam as posições 1 a 718 na mesma ordem do arquivo, o total previsto é $\sum_{i=1}^{718} i + 438 = 258.121 + 438 = 258.559$, precisamente o valor medido.

13.4 Aderência quantitativa da Solução 2

Nas 28 execuções da Solução 2 nos Experimentos A e C, as contagens da ordenação e da busca ficaram dentro das faixas teóricas. Com entradas embaralhadas, o MergeSort ficou próximo do pior caso (entre 88% e 100% do máximo teórico, e acima de 94% para $N \geq 40$), como se espera para dados em ordem aleatória. A busca binária ficou no interior da sua faixa estreita, de largura N . No benchmark, a razão entre comparações totais e $N \lg N$ ficou entre 1,88 e 1,93 para N de 1.000 a 512.000, estável e próxima do valor 2 previsto para $Q = 0$ (a diferença vem dos termos lineares negativos de $W(n)$).

No Experimento C, o efeito de Q é logarítmico, e não linear como na Solução 1: passar de $Q = 80$ para $Q = 160$ acrescenta cerca de 770 comparações, das quais a maior parte vem da ordenação de D ($\Theta(Q \lg Q)$) e apenas 184 das buscas, que ganham cerca de uma iteração cada quando Q dobra. Na Solução 1, o mesmo acréscimo de Q custou 12.800 comparações.

13.5 O efeito da quase-ordenação do arquivo original

No Experimento D, o MergeSort recursivo fez 3.353 comparações, contra 5.921 com os mesmos nomes embaralhados, e ficou próximo do melhor caso $B(718) = 3.267$. Isso acontece porque o arquivo original é formado por dois blocos já ordenados (Seção 2.3): quase todas as intercalações esgotam um dos lados cedo. A ordem dos dados altera a constante, mas não a classe $\Theta(n \lg n)$, exatamente como previsto. A duplicata real foi detectada pelos quatro modos, e os destinos gerados são idênticos aos da Solução 1.

13.6 Comparação de tempo entre iterativa e recursiva

Busca sequencial. As duas versões fazem as mesmas comparações, mas a recursiva foi de 2,8 a 3,1 vezes mais lenta no benchmark e cerca de 2,9 vezes mais lenta com os dados reais para $N = 718$. A razão é que a recursiva executa uma chamada de função por elemento visitado (empilhar argumentos e endereço de retorno, saltar, retornar), e esse custo é comparável ao de uma comparação de nomes que geralmente difere logo nos primeiros caracteres. É uma diferença de constante, não de classe de complexidade.

A diferença estrutural mais relevante permanece no espaço auxiliar, e ela também foi observada na prática: com a pilha padrão de 8 MB do Linux, a busca recursiva executou normalmente com um destino de 100.000 registros, mas o programa foi encerrado por estouro de pilha (falha de segmentação) com 200.000. A versão iterativa não tem esse limite. No Windows, cuja pilha padrão é de 1 MB, o limite seria cerca de oito vezes menor.

MergeSort e busca binária. Como previsto, as diferenças são pequenas. No MergeSort, as duas versões fazem praticamente o mesmo trabalho (a recursiva faz cerca de $2n$ chamadas no total, desprezíveis diante das $n \lg n$ comparações), e a recursiva chegou a ser um pouco mais rápida para N grande, provavelmente pela melhor localidade de acesso da divisão *top-down*. Na busca binária recursiva há apenas cerca de $\lg n$ chamadas por busca, e a fase de busca foi entre 2% e 18% mais lenta que a iterativa. A pilha usada por ela é logarítmica e nunca representa risco.

13.7 Crescimento do tempo

Solução 1. O ajuste por mínimos quadrados do logaritmo do tempo contra $\lg N$, no benchmark, dá inclinação 2,026 na versão iterativa e 2,061 na recursiva, compatíveis com $\Theta(N^2)$. As razões $t(2N)/t(N)$ medidas foram 4,03, 4,10 e 4,08 (iterativa), próximas do valor 4 previsto pela Equação (3). Diferentemente das contagens, os tempos têm ruído (escalonamento, *cache*, frequência do processador), por isso a concordância é aproximada.

Solução 2. As razões $t(2N)/t(N)$ ficaram entre 2,2 e 2,6, e a inclinação log-log foi 1,22 na versão iterativa e 1,21 na recursiva. Para $N \lg N$ nessa faixa de N , a inclinação teórica local é $1 + 1/\ln N$, entre 1,08 e 1,14. O tempo cresce, portanto, um pouco mais depressa que as comparações: o tempo por unidade de $N \lg N$ subiu de 25 ns para 54 ns. Isso não contradiz a análise, que conta comparações no modelo em que acessar qualquer posição da memória custa o mesmo. Na máquina real, à medida que N cresce, os vetores e as cadeias deixam de caber na memória *cache*, e a busca binária e a intercalação acessam posições distantes, cada vez mais lentas. O número de operações segue $\Theta(N \lg N)$; o custo de cada operação é que aumenta.

13.8 Solução 1 × Solução 2

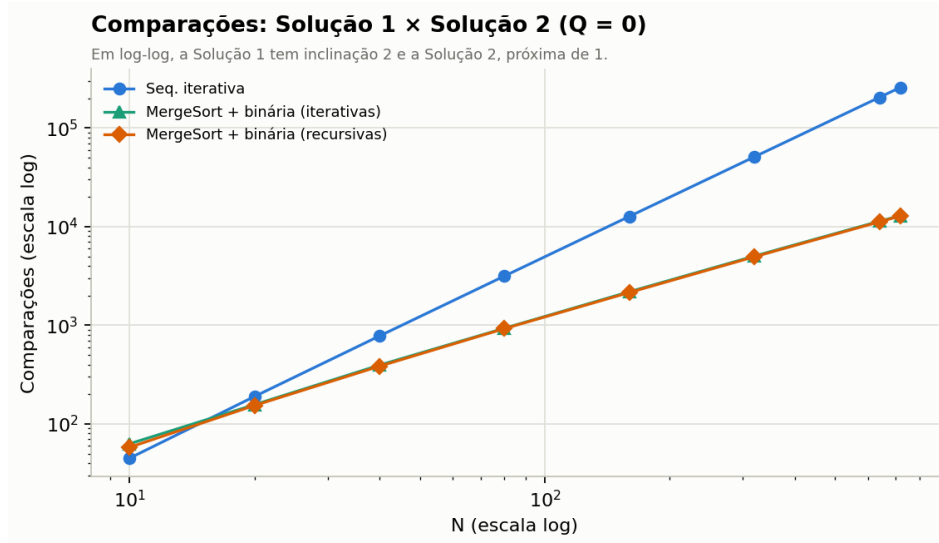


Figura 8: Comparações das duas soluções com os dados reais, em escala log-log: inclinação 2 para a Solução 1 e próxima de 1 para a Solução 2.

A teoria previu que a Solução 1 poderia fazer menos comparações para N muito pequeno (cruzamento em torno de $N \approx 16$). De fato, com $N = 10$ a Solução 1 fez 45 comparações e a Solução 2, de 58 a 63; com $N = 20$ a Solução 2 já faz menos (154 a 157, contra 190). Com os dados reais, a Solução 2 passa a ser mais rápida em tempo a partir de $N \approx 80$; para $N = 718$, faz 20 vezes menos comparações e é 4,9 vezes mais rápida, pois parte do tempo é gasto em leitura e escrita, comum às duas.

No benchmark, para $N = 8.000$, a Solução 1 fez 162 vezes mais comparações que a Solução 2 (31.996.000 contra 197.558) e foi 60 vezes mais lenta. A diferença entre os fatores 162 e 60 reflete o custo por comparação: a busca sequencial percorre o vetor em ordem, o que é favorável à *cache*, enquanto a Solução 2 acessa posições distantes e ainda copia dados na intercalação.

14 Discussão dos resultados

Os dados empíricos confirmam as hipóteses de complexidade temporal propostas? Sim. Na Solução 1, em 28 execuções sob as hipóteses da análise, nos dois modos de busca, a diferença entre previsto e medido foi zero em todas; a inclinação do ajuste log-log das comparações é 2,0198 e a razão $T(2N)/T(N)$ converge para 4, ambas assinaturas de crescimento quadrático. Na Solução 2, todas as contagens ficaram dentro dos limites teóricos, e a razão entre comparações e $N \lg N$ permaneceu estável.

Existem diferenças entre valores teóricos e medidos? Nas contagens de comparações da Solução 1 não há diferença alguma sob (H2). Isso se explica pela natureza da grandeza medida: o contador registra um evento discreto do algoritmo, cada execução de `strcmp`, e não um intervalo de tempo; não há, portanto, fontes de ruído. Há diferença, sim, no Experimento D: 438 comparações acima do previsto, causada pela violação deliberada da hipótese (H2), e não por erro do modelo. Na Solução 2, a teoria fornece faixas e não um valor exato, porque o número de comparações do MergeSort depende da ordem dos dados. Nos tempos há diferenças pequenas, esperadas pelo ruído de medição e, na Solução 2, pelo efeito da hierarquia de memória.

Iterativa e recursiva produzem o mesmo resultado? Em número de comparações da busca, sim, em todos os experimentos: as versões percorrem os dados na mesma ordem e param no mesmo ponto. Os MergeSorts diferem ligeiramente nas contagens, porque dividem o vetor de formas diferentes, mas ambos são $\Theta(n \lg n)$. A diferença está no espaço (pilha $\Theta(n)$ na busca sequencial recursiva e $\Theta(\lg n)$

nas recursões da Solução 2) e no tempo, que é cerca de 3 vezes maior na busca sequencial recursiva e quase igual nas demais.

Os resultados sustentam a demonstração matemática? Sustentam, com a ressalva de que experimentos não demonstram teoremas: a prova por indução estabelece a fórmula para todo $N \geq 1$ por dedução lógica. O que os experimentos fazem é validar que a modelagem do custo de cada inserção está correta e que o programa implementado, em todas as variantes, é de fato o algoritmo analisado.

Implicação prática. A classificação $\Theta(N^2)$ diz que a Solução 1 não escala: para os 718 nomes deste arquivo, a execução é instantânea, mas um cadastro de 100.000 bolsistas exigiria cerca de 5×10^9 comparações, e 1.000.000 levaria a 5×10^{11} , inviável na prática. A Solução 2 faria cerca de 4×10^7 comparações para um milhão de registros. O gargalo está na estrutura de dados e no algoritmo de busca, não na linguagem, na máquina, nem na escolha entre iteração e recursão. Uma tabela de dispersão reduziria ainda mais o custo esperado, para $O(N + Q)$.

15 Conclusão

O trabalho implementou em C o serviço de importação de bolsistas com detecção de redundância na coluna Nome, usando o arquivo `bolsistas-2025.csv` do Portal de Dados Abertos, em quatro variantes: busca sequencial iterativa e recursiva (Solução 1) e MergeSort seguido de busca binária, ambos em versões iterativa e recursiva (Solução 2). O algoritmo foi instrumentado com contadores de comparações, e a análise relacionou as funções deduzidas, as demonstrações formais, os dados medidos e as classificações assintóticas.

Na Solução 1, a função deduzida foi $T(N, Q) = N \cdot Q + N(N - 1)/2$, obtida somando os custos individuais das N inserções, cada uma percorrendo um vetor de $Q + i - 1$ chaves, e demonstrada por indução para todo $N \geq 1$. Os dados confirmaram a fórmula sem nenhuma margem de erro nas 28 execuções sob a hipótese de distinção, e a única divergência, no experimento com o arquivo completo, foi explicada exatamente pelo encerramento antecipado da busca na posição 438. As versões iterativa e recursiva fazem as mesmas comparações, mas diferem no espaço auxiliar ($\Theta(1)$ contra $\Theta(n)$) e no tempo: a recursiva foi cerca de 3 vezes mais lenta e falhou por estouro de pilha com 200.000 registros.

Na Solução 2, o MergeSort tem custo $\Theta(n \lg n)$ em qualquer entrada e espaço auxiliar $\Theta(n)$ nas duas versões, com $\Theta(\lg n)$ de pilha adicional na recursiva; a busca binária tem custo $\Theta(\lg n)$, com espaço $\Theta(1)$ na iterativa e $\Theta(\lg n)$ na recursiva. O custo total é $\Theta((N + Q) \lg(N + Q))$. Todas as contagens ficaram dentro dos limites teóricos, e a quase-ordenação do arquivo original levou o MergeSort para perto do melhor caso, como previsto. O tempo cresceu um pouco mais rápido que $N \lg N$ por efeito da hierarquia de memória, fator que o modelo de contagem de comparações não considera.

A classificação assintótica da Solução 1 é $\Theta(N^2)$ para Q constante, e a da Solução 2, $\Theta(N \lg N)$. A Solução 2 foi 60 vezes mais rápida para 8.000 registros, enquanto para cadastros muito pequenos a Solução 1 é competitiva. A lição que fica é que a Solução 1 está correta e é inadequada ao mesmo tempo: resolve o problema especificado, mas não escala; a intervenção necessária está na estrutura de dados e no algoritmo que sustentam a verificação de redundância, e não na escolha entre iteração e recursão, que altera apenas constantes e o uso de pilha.

16 Repositório Git

Todo o material deste trabalho está versionado em repositório Git público:

<https://github.com/owhska/insercao-sem-duplicatas-csv>

O repositório contém:

- `src/algoritmos.c` e `src/algoritmos.h`: buscas sequenciais, MergeSorts, buscas binárias e contadores;

- `src/copiador.c`: programa principal; `src/bench.c`: benchmark em memória;
- `dados/bolsistas-2025.csv`: arquivo de entrada original;
- `experimentos/entradas/`: arquivos utilizados nos testes;
- `experimentos/estatisticas/`: arquivos CSV gerados nos experimentos;
- `experimentos/destinos/` e `experimentos/saidas/`: arquivos de destino e saídas padrão produzidas pelo programa;
- `Makefile`, `preparar_entradas.py` e `rodar_experimentos.sh`: compilação, geração das entradas e execução da bateria completa;
- `analisar.py`: confronto com o modelo teórico e geração dos gráficos;
- `graficos/`: gráficos deste relatório;
- `README.md`: instruções de compilação e de execução;
- `relatorio/`: fonte e PDF deste relatório.

Compilação:

```
make
```

Execução:

```
./src/copiador dados/bolsistas-2025.csv cadastro.csv
./src/copiador dados/bolsistas-2025.csv cadastro.csv bin-rec estatisticas.csv
./rodar_experimentos.sh && python3 analisar.py
```

A Código-fonte completo

Listing 6: *algoritmos.h*

```
1 #ifndef ALGORITMOS_H
2 #define ALGORITMOS_H
3
4 #include <stddef.h>
5
6 extern unsigned long long comp_ordenacao;
7 extern unsigned long long comp_busca;
8
9 int busca_sequencial_iterativa(char **v, size_t n, const char *chave);
10 int busca_sequencial_rekursiva(char **v, size_t n, const char *chave);
11
12 void mergesort_iterativo(char **v, size_t n);
13 void mergesort_rekursivo(char **v, size_t n);
14
15 size_t busca_binaria_iterativa(char **v, size_t n, const char *chave);
16 size_t busca_binaria_rekursiva(char **v, size_t n, const char *chave);
17
18 double agora_segundos(void);
19
20 #endif
```

Listing 7: *algoritmos.c*

```
1 #define _POSIX_C_SOURCE 199309L
2
3 #include <stdio.h>
4 #include <stdlib.h>
5 #include <string.h>
6 #include <time.h>
7 #include "algoritmos.h"
8
```

```

9  #if defined(_WIN32) || defined(_WIN64)
10 #include <windows.h>
11 double agora_segundos(void) {
12     LARGE_INTEGER freq, cont;
13     QueryPerformanceFrequency(&freq);
14     QueryPerformanceCounter(&cont);
15     return (double)cont.QuadPart / (double)freq.QuadPart;
16 }
17 #else
18 double agora_segundos(void) {
19     struct timespec ts;
20     clock_gettime(CLOCK_MONOTONIC, &ts);
21     return (double)ts.tv_sec + (double)ts.tv_nsec / 1e9;
22 }
23 #endif
24
25 unsigned long long comp_ordenacao = 0;
26 unsigned long long comp_busca = 0;
27
28 int busca_sequencial_iterativa(char **v, size_t n, const char *chave) {
29     size_t i;
30     for (i = 0; i < n; i++) {
31         comp_busca++;
32         if (strcmp(v[i], chave) == 0) return 1;
33     }
34     return 0;
35 }
36
37 static int busca_sequencial_rec_aux(char **v, size_t n, const char *chave, size_t i) {
38     if (i >= n) return 0;
39     comp_busca++;
40     if (strcmp(v[i], chave) == 0) return 1;
41     return busca_sequencial_rec_aux(v, n, chave, i + 1);
42 }
43
44 int busca_sequencial_recurativa(char **v, size_t n, const char *chave) {
45     return busca_sequencial_rec_aux(v, n, chave, 0);
46 }
47
48 static void intercalar(char **v, char **aux, size_t ini, size_t meio, size_t fim) {
49     size_t i = ini, j = meio, k = ini;
50     while (i < meio && j < fim) {
51         comp_ordenacao++;
52         if (strcmp(v[i], v[j]) <= 0) aux[k++] = v[i++];
53         else aux[k++] = v[j++];
54     }
55     while (i < meio) aux[k++] = v[i++];
56     while (j < fim) aux[k++] = v[j++];
57     memcpy(v + ini, aux + ini, (fim - ini) * sizeof(char *));
58 }
59
60 static void mergesort_rec_aux(char **v, char **aux, size_t ini, size_t fim) {
61     size_t meio;
62     if (fim - ini < 2) return;
63     meio = ini + (fim - ini) / 2;
64     mergesort_rec_aux(v, aux, ini, meio);
65     mergesort_rec_aux(v, aux, meio, fim);
66     intercalar(v, aux, ini, meio, fim);
67 }
68
69 static char **alocar_auxiliar(size_t n) {
70     char **aux = malloc((n ? n : 1) * sizeof(char *));
71     if (!aux) {
72         fprintf(stderr, "ERRO: memoria insuficiente para o MergeSort.\n");
73         exit(EXIT_FAILURE);
74     }
75     return aux;
76 }
77
78 void mergesort_recurativo(char **v, size_t n) {
79     char **aux;
80     if (n < 2) return;
81     aux = alocar_auxiliar(n);

```

```

82     mergesort_rec_aux(v, aux, 0, n);
83     free(aux);
84 }
85
86 void mergesort_iterativo(char **v, size_t n) {
87     char **aux;
88     size_t largura, ini, meio, fim;
89     if (n < 2) return;
90     aux = alocar_auxiliar(n);
91     for (largura = 1; largura < n; largura *= 2) {
92         for (ini = 0; ini + largura < n; ini += 2 * largura) {
93             meio = ini + largura;
94             fim = (ini + 2 * largura < n) ? ini + 2 * largura : n;
95             intercalar(v, aux, ini, meio, fim);
96         }
97     }
98     free(aux);
99 }
100
101 size_t busca_binaria_iterativa(char **v, size_t n, const char *chave) {
102     size_t lo = 0, hi = n, meio;
103     while (lo < hi) {
104         meio = lo + (hi - lo) / 2;
105         comp_busca++;
106         if (strcmp(v[meio], chave) < 0) lo = meio + 1;
107         else hi = meio;
108     }
109     return lo;
110 }
111
112 static size_t busca_binaria_rec_aux(char **v, const char *chave, size_t lo, size_t hi) {
113     size_t meio;
114     if (lo >= hi) return lo;
115     meio = lo + (hi - lo) / 2;
116     comp_busca++;
117     if (strcmp(v[meio], chave) < 0) return busca_binaria_rec_aux(v, chave, meio + 1, hi);
118     return busca_binaria_rec_aux(v, chave, lo, meio);
119 }
120
121 size_t busca_binaria_recursiva(char **v, size_t n, const char *chave) {
122     return busca_binaria_rec_aux(v, chave, 0, n);
123 }

```

Listing 8: *copiador.c*

```

1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4 #include "algoritmos.h"
5
6 #define MAX_LINHA 4096
7 #define COLUNA_CHAVE 0
8
9 typedef struct {
10     char **itens;
11     size_t qtd;
12     size_t capacidade;
13 } Vetor;
14
15 typedef enum { SEQ_ITER, SEQ_REC, BIN_ITER, BIN_REC } Modo;
16
17 static const char *nomes_modos[] = { "seq-iter", "seq-rec", "bin-iter", "bin-rec" };
18
19 static void vetor_iniciar(Vetor *v) {
20     v->itens = NULL;
21     v->qtd = 0;
22     v->capacidade = 0;
23 }
24
25 static void vetor_inserir(Vetor *v, char *s) {
26     if (v->qtd == v->capacidade) {
27         size_t nova = v->capacidade ? v->capacidade * 2 : 16;
28         char **bloco = realloc(v->itens, nova * sizeof(char *));

```

```

29     if (!bloco) {
30         fprintf(stderr, "ERRO: memoria insuficiente.\n");
31         exit(EXIT_FAILURE);
32     }
33     v->itens = bloco;
34     v->capacidade = nova;
35 }
36 v->itens[v->qtd++] = s;
37 }
38
39 static void vetor_liberar(Vetor *v, int liberar_itens) {
40     size_t i;
41     if (liberar_itens)
42         for (i = 0; i < v->qtd; i++) free(v->itens[i]);
43     free(v->itens);
44     vetor_iniciar(v);
45 }
46
47 static char *duplicar(const char *s) {
48     char *d = malloc(strlen(s) + 1);
49     if (!d) {
50         fprintf(stderr, "ERRO: memoria insuficiente.\n");
51         exit(EXIT_FAILURE);
52     }
53     strcpy(d, s);
54     return d;
55 }
56
57 static void remover_fim_de_linha(char *linha) {
58     size_t n = strlen(linha);
59     while (n > 0 && (linha[n - 1] == '\n' || linha[n - 1] == '\r')) linha[--n] = '\0';
60 }
61
62 static int linha_vazia(const char *linha) {
63     for (; *linha; linha++)
64         if (*linha != ' ' && *linha != '\t') return 0;
65     return 1;
66 }
67
68 static char *extrair_chave(const char *linha) {
69     int coluna = 0;
70     const char *ini = linha, *p;
71     char *chave;
72     size_t tam;
73     while (coluna < COLUNA_CHAVE && *ini) {
74         if (*ini == ',') coluna++;
75         ini++;
76     }
77     for (p = ini; *p && *p != ','; p++);
78     tam = (size_t)(p - ini);
79     chave = malloc(tam + 1);
80     if (!chave) {
81         fprintf(stderr, "ERRO: memoria insuficiente.\n");
82         exit(EXIT_FAILURE);
83     }
84     memcpy(chave, ini, tam);
85     chave[tam] = '\0';
86     return chave;
87 }
88
89 static size_t carregar_chaves_destino(const char *caminho, Vetor *chaves) {
90     FILE *fp = fopen(caminho, "r");
91     char linha[MAX_LINHA];
92     int primeira = 1;
93     if (!fp) return 0;
94     while (fgets(linha, sizeof(linha), fp)) {
95         remover_fim_de_linha(linha);
96         if (primeira) { primeira = 0; continue; }
97         if (linha_vazia(linha)) continue;
98         vetor_inserir(chaves, extrair_chave(linha));
99     }
100     fclose(fp);
101     return chaves->qtd;

```

```

102 }
103
104 static int destino_vazio(const char *caminho) {
105     FILE *fp = fopen(caminho, "r");
106     long tam;
107     if (!fp) return 1;
108     fseek(fp, 0, SEEK_END);
109     tam = ftell(fp);
110     fclose(fp);
111     return tam <= 0;
112 }
113
114 static void garantir_quebra_final(const char *caminho) {
115     FILE *fp = fopen(caminho, "rb");
116     int ultimo;
117     if (!fp) return;
118     if (fseek(fp, -1, SEEK_END) != 0) { fclose(fp); return; }
119     ultimo = fgetc(fp);
120     fclose(fp);
121     if (ultimo != '\n') {
122         fp = fopen(caminho, "a");
123         if (fp) { fputc('\n', fp); fclose(fp); }
124     }
125 }
126
127 static void gravar_estatisticas(const char *caminho, Modo modo, size_t lidos, size_t previos,
128                                size_t inseridos, size_t redundantes,
129                                double t_ord, double t_busca, double t_total) {
130     FILE *fp = fopen(caminho, "a+");
131     if (!fp) { perror("ERRO ao abrir estatisticas"); return; }
132     fseek(fp, 0, SEEK_END);
133     if (ftell(fp) == 0)
134         fprintf(fp, "modo,lidos,previos,inseridos,redundantes,comp_ordenacao,comp_busca,
135 comp_total,tempo_ordenacao_s,tempo_verificacao_s,tempo_total_s\n");
136     fprintf(fp, "%s,%zu,%zu,%zu,%zu,%llu,%llu,%llu,%.9f,%.9f,%.9f\n",
137             nomes_modos[modo], lidos, previos, inseridos, redundantes,
138             comp_ordenacao, comp_busca, comp_ordenacao + comp_busca, t_ord, t_busca, t_total);
139     fclose(fp);
140 }
141
142 static void uso(const char *prog) {
143     fprintf(stderr, "Uso: %s <novos.csv> <destino.csv> [seq-iter|seq-rec|bin-iter|bin-rec] [
144 estatisticas.csv]\n", prog);
145 }
146
147 int main(int argc, char *argv[]) {
148     Modo modo = SEQ_ITER;
149     Vetor destino_chaves, linhas, chaves_novas, ordenadas;
150     FILE *origem, *destino;
151     char linha[MAX_LINHA];
152     char *cabecalho = NULL;
153     int primeira = 1;
154     size_t previos, lidos = 0, inseridos = 0, redundantes = 0, i;
155     double t0, t1, t2;
156     int m;
157
158     if (argc < 3 || argc > 5) { uso(argv[0]); return EXIT_FAILURE; }
159     if (argc >= 4) {
160         for (m = 0; m < 4; m++)
161             if (strcmp(argv[3], nomes_modos[m]) == 0) break;
162         if (m == 4) { uso(argv[0]); return EXIT_FAILURE; }
163         modo = (Modo)m;
164     }
165
166     vetor_iniciar(&destino_chaves);
167     vetor_iniciar(&linhas);
168     vetor_iniciar(&chaves_novas);
169     vetor_iniciar(&ordenadas);
170
171     origem = fopen(argv[1], "r");
172     if (!origem) { perror("ERRO ao abrir arquivo de novos registros"); return EXIT_FAILURE; }
173     while (fgets(linha, sizeof(linha), origem)) {
174         remover_fim_de_linha(linha);
175     }

```

```

173     if (primeira) { primeira = 0; cabecalho = duplicar(linha); continue; }
174     if (linha_vazia(linha)) continue;
175     vetor_inserir(&linhas, duplicar(linha));
176     vetor_inserir(&chaves_novas, extrair_chave(linha));
177 }
178 fclose(origem);
179 lidos = linhas.qtd;
180
181 if (destino_vazio(argv[2])) {
182     destino = fopen(argv[2], "w");
183     if (!destino) { perror("ERRO ao criar destino"); return EXIT_FAILURE; }
184     fprintf(destino, "%s\n", cabecalho ? cabecalho : "");
185     fclose(destino);
186 } else {
187     garantir_quebra_final(argv[2]);
188 }
189 previos = carregar_chaves_destino(argv[2], &destino_chaves);
190
191 destino = fopen(argv[2], "a");
192 if (!destino) { perror("ERRO ao abrir destino"); return EXIT_FAILURE; }
193
194 t0 = agora_segundos();
195 t1 = t0;
196
197 if (modo == SEQ_ITER || modo == SEQ_REC) {
198     for (i = 0; i < lidos; i++) {
199         char *chave = chaves_novas.itens[i];
200         int existe = (modo == SEQ_ITER)
201             ? busca_sequencial_iterativa(destino_chaves.itens, destino_chaves.qtd, chave)
202             : busca_sequencial_recurativa(destino_chaves.itens, destino_chaves.qtd, chave);
203         if (existe) {
204             printf("ERRO: valor duplicado na chave, registro ignorado: %s\n", chave);
205             redundantes++;
206             continue;
207         }
208         fprintf(destino, "%s\n", linhas.itens[i]);
209         vetor_inserir(&destino_chaves, duplicar(chave));
210         inseridos++;
211     }
212 } else {
213     for (i = 0; i < lidos; i++) vetor_inserir(&ordenadas, chaves_novas.itens[i]);
214     if (modo == BIN_ITER) {
215         mergesort_iterativo(destino_chaves.itens, destino_chaves.qtd);
216         mergesort_iterativo(ordenadas.itens, ordenadas.qtd);
217     } else {
218         mergesort_recurativo(destino_chaves.itens, destino_chaves.qtd);
219         mergesort_recurativo(ordenadas.itens, ordenadas.qtd);
220     }
221     t1 = agora_segundos();
222     for (i = 0; i < lidos; i++) {
223         char *chave = chaves_novas.itens[i];
224         size_t q = destino_chaves.qtd, pos;
225         int existe = 0;
226         if (q > 0) {
227             pos = (modo == BIN_ITER)
228                 ? busca_binaria_iterativa(destino_chaves.itens, q, chave)
229                 : busca_binaria_recurativa(destino_chaves.itens, q, chave);
230             if (pos < q) {
231                 comp_busca++;
232                 existe = (strcmp(destino_chaves.itens[pos], chave) == 0);
233             }
234         }
235         if (!existe) {
236             pos = (modo == BIN_ITER)
237                 ? busca_binaria_iterativa(ordenadas.itens, ordenadas.qtd, chave)
238                 : busca_binaria_recurativa(ordenadas.itens, ordenadas.qtd, chave);
239             existe = (ordenadas.itens[pos] != chave);
240         }
241         if (existe) {
242             printf("ERRO: valor duplicado na chave, registro ignorado: %s\n", chave);
243             redundantes++;
244             continue;
245         }
246     }

```

```

246         fprintf(destino, "%s\n", linhas[itens[i]]);
247         inseridos++;
248     }
249 }
250
251 t2 = agora_segundos();
252 fclose(destino);
253
254 printf("\n");
255 printf("Modo                : %s\n", nomes_modos[modo]);
256 printf("Registros lidos      : %zu\n", lidos);
257 printf("Registros previos no destino: %zu\n", previos);
258 printf("Registros inseridos    : %zu\n", inseridos);
259 printf("Registros duplicados    : %zu\n", redundantes);
260 printf("Comparacoes na ordenacao : %llu\n", comp_ordenacao);
261 printf("Comparacoes na busca     : %llu\n", comp_busca);
262 printf("Comparacoes totais       : %llu\n", comp_ordenacao + comp_busca);
263 printf("Tempo total              : %.6f s\n", t2 - t0);
264
265 if (argc == 5)
266     gravar_estatisticas(argv[4], modo, lidos, previos, inseridos, redundantes,
267                         t1 - t0, t2 - t1, t2 - t0);
268
269 free(cabecalho);
270 vetor_liberar(&destino_chaves, 1);
271 vetor_liberar(&linhas, 1);
272 vetor_liberar(&chaves_novas, 1);
273 vetor_liberar(&ordenadas, 0);
274 return EXIT_SUCCESS;
275 }

```

Listing 9: *bench.c*

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4  #include "algoritmos.h"
5
6  #define MAX_LINHA 4096
7  #define MAX_NOMES 100000
8
9  static char *nomes_base[MAX_NOMES];
10 static size_t qtd_base = 0;
11 static unsigned long long semente = 20260923ULL;
12
13 static unsigned long long aleatorio(void) {
14     semente = semente * 6364136223846793005ULL + 1442695040888963407ULL;
15     return semente >> 33;
16 }
17
18 static void carregar_base(const char *caminho) {
19     FILE *fp = fopen(caminho, "r");
20     char linha[MAX_LINHA];
21     int primeira = 1;
22     size_t i;
23     if (!fp) { perror("ERRO ao abrir base"); exit(EXIT_FAILURE); }
24     while (fgets(linha, sizeof(linha), fp) && qtd_base < MAX_NOMES) {
25         char *p = strchr(linha, ',');
26         int repetido = 0;
27         if (primeira) { primeira = 0; continue; }
28         if (p) *p = '\0';
29         linha[strcspn(linha, "\r\n")] = '\0';
30         if (!linha[0]) continue;
31         for (i = 0; i < qtd_base; i++)
32             if (strcmp(nomes_base[i], linha) == 0) { repetido = 1; break; }
33         if (repetido) continue;
34         nomes_base[qtd_base] = malloc(strlen(linha) + 1);
35         strcpy(nomes_base[qtd_base++], linha);
36     }
37     fclose(fp);
38 }
39
40 static char **gerar_chaves(size_t n) {

```

```

41     char **v = malloc(n * sizeof(char *));
42     size_t i, j;
43     for (i = 0; i < n; i++) {
44         char buf[MAX_LINHA];
45         snprintf(buf, sizeof(buf), "%s #%zu", nomes_base[i % qtd_base], i / qtd_base);
46         v[i] = malloc(strlen(buf) + 1);
47         strcpy(v[i], buf);
48     }
49     for (i = n - 1; i > 0; i--) {
50         char *t;
51         j = (size_t)(aleatorio() % (i + 1));
52         t = v[i]; v[i] = v[j]; v[j] = t;
53     }
54     return v;
55 }
56
57 static int cmp_double(const void *a, const void *b) {
58     double x = *(const double *)a, y = *(const double *)b;
59     return (x > y) - (x < y);
60 }
61
62 static void rodar_sequencial(char **chaves, size_t n, int recursiva, double *t, unsigned long
63                             long *cb) {
64     char **cad = malloc(n * sizeof(char *));
65     size_t qtd = 0, i;
66     double t0 = agora_segundos();
67     comp_busca = 0;
68     for (i = 0; i < n; i++) {
69         int existe = recursiva ? busca_sequencial_recursiva(cad, qtd, chaves[i])
70                               : busca_sequencial_iterativa(cad, qtd, chaves[i]);
71         if (!existe) cad[qtd++] = chaves[i];
72     }
73     *t = agora_segundos() - t0;
74     *cb = comp_busca;
75     free(cad);
76 }
77
78 static void rodar_binaria(char **chaves, size_t n, int recursiva, double *t_ord, double *t_bus
79                           ,
80                           unsigned long long *co, unsigned long long *cb) {
81     char **ord = malloc(n * sizeof(char *));
82     size_t i, inseridos = 0, pos;
83     double t0, t1, t2;
84     memcpy(ord, chaves, n * sizeof(char *));
85     comp_ordenacao = 0;
86     comp_busca = 0;
87     t0 = agora_segundos();
88     if (recursiva) mergesort_recursivo(ord, n); else mergesort_iterativo(ord, n);
89     t1 = agora_segundos();
90     for (i = 0; i < n; i++) {
91         pos = recursiva ? busca_binaria_recursiva(ord, n, chaves[i])
92                       : busca_binaria_iterativa(ord, n, chaves[i]);
93         if (ord[pos] == chaves[i]) inseridos++;
94     }
95     t2 = agora_segundos();
96     if (inseridos != n) fprintf(stderr, "AVISO: inseridos=%zu n=%zu\n", inseridos, n);
97     *t_ord = t1 - t0;
98     *t_bus = t2 - t1;
99     *co = comp_ordenacao;
100    *cb = comp_busca;
101    free(ord);
102 }
103
104 int main(int argc, char *argv[]) {
105     size_t tam_seq[] = { 1000, 2000, 4000, 8000 };
106     size_t tam_bin[] = { 1000, 2000, 4000, 8000, 16000, 32000, 64000, 128000, 256000, 512000
107     };
108     int reps = 15, r, modo;
109     size_t k, i;
110     double tempos[64], tempos_b[64];
111     if (argc < 2) { fprintf(stderr, "Uso: %s <base.csv> [repeticoes]\n", argv[0]); return
112     EXIT_FAILURE; }
113     if (argc >= 3) reps = atoi(argv[2]);

```

```

110     if (reps < 1 || reps > 64) reps = 15;
111     carregar_base(argv[1]);
112     printf("solucao,modo,n,comp_ordenacao,comp_busca,tempo_ordenacao_s,tempo_busca_s,
113     tempo_total_s\n");
114     for (k = 0; k < sizeof(tam_seq) / sizeof(tam_seq[0]); k++) {
115         size_t n = tam_seq[k];
116         char **chaves = gerar_chaves(n);
117         for (modo = 0; modo < 2; modo++) {
118             unsigned long long cb = 0;
119             for (r = 0; r < reps; r++) rodar_sequencial(chaves, n, modo, &tempos[r], &cb);
120             qsort(tempos, (size_t)reps, sizeof(double), cmp_double);
121             printf("1,%s,%zu,0,%llu,0,%.9f,%.9f\n", modo ? "seq-rec" : "seq-iter", n, cb,
122                 tempos[reps / 2], tempos[reps / 2]);
123             fflush(stdout);
124         }
125         for (i = 0; i < n; i++) free(chaves[i]);
126         free(chaves);
127     }
128     for (k = 0; k < sizeof(tam_bin) / sizeof(tam_bin[0]); k++) {
129         size_t n = tam_bin[k];
130         char **chaves = gerar_chaves(n);
131         for (modo = 0; modo < 2; modo++) {
132             unsigned long long co = 0, cb = 0;
133             double tt[64];
134             for (r = 0; r < reps; r++) {
135                 rodar_binaria(chaves, n, modo, &tempos[r], &tempos_b[r], &co, &cb);
136                 tt[r] = tempos[r] + tempos_b[r];
137             }
138             qsort(tempos, (size_t)reps, sizeof(double), cmp_double);
139             qsort(tempos_b, (size_t)reps, sizeof(double), cmp_double);
140             qsort(tt, (size_t)reps, sizeof(double), cmp_double);
141             printf("2,%s,%zu,%llu,%llu,%.9f,%.9f,%.9f\n", modo ? "bin-rec" : "bin-iter", n, co
142                 , cb,
143                 tempos[reps / 2], tempos_b[reps / 2], tt[reps / 2]);
144             fflush(stdout);
145         }
146         for (i = 0; i < n; i++) free(chaves[i]);
147         free(chaves);
148     }
149     return EXIT_SUCCESS;
150 }

```